

You are a code translator. Please translate the following correct Python code into Rust, ensuring the same functionality and correctness. Keep the structure idiomatic for Rust. Do not simplify or alter logic. Here is the Python code:

python

🔼 折叠运行复制

```
class Solution:
    precedence_stack = {
        '(': 0,
        '+': 2,
        '-': 2,
        '*': 4,
        '/': 4,
        'neg': 5,
    }
    precedence_input = {
        '(': 6,
        ')': 0,
        '+': 1,
        '-': 1,
        '*': 3,
        '/': 3,
        'neg': 5,
    }
    operators = set("+-*/")
    def transform(self, s):
        # The generated postfix expression of s
        output = []
        # Wrap `s` with '(' and ')' to make processing easier
        s = s + ')'
        stack = ['(']
        prevC = '('
        hasNum = False
        num = 0
        for c in s:
            if c == ' ':
                continue
            # c is part of number
            if c.isdigit():
                hasNum = True
```

```

        num = num * 10 + int(c)

        prevC = c

        continue

# c is operator
if hasNum:
    hasNum = False

    output.append(num)

    num = 0

# check if is negative unary operator
if c == '-' and (prevC == '(' or prevC in self.operators):
    c = 'neg'

# Pop until '('
while self.precedence_stack[stack[-1]] >= self.precedence_input[c]:
    popped = stack.pop()

    if popped != '(':
        output.append(popped)

    else:
        break

# push input on to the stack
if c != ')':
    stack.append(c)

    prevC = c

return output

def evaluate_postfix(self, postfix):
    stack = []

    for each in postfix:
        # Is number
        if each not in self.precedence_input:
            stack.append(each)

            continue

        # Is operator
        if each == '+':
            stack.append(stack.pop() + stack.pop())

        elif each == '-':
            a, b = stack.pop(), stack.pop()

            stack.append(b - a)

        elif each == '*':
            stack.append(stack.pop() * stack.pop())

        elif each == '/':
            a, b = stack.pop(), stack.pop()

            stack.append(b / a)

```

```

        elif each == '/':
            a, b = stack.pop(), stack.pop()
            stack.append(int(b / a)) # truncate towards zero
        elif each == 'neg':
            stack.append(-stack.pop())
    return stack[0]

def calculate(self, s):
    postfix = self.transform(s)
    return self.evaluate_postfix(postfix)

```

HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

rust

🔼 折叠复制

```

use std::collections::HashMap;

struct Solution;

impl Solution {
    fn precedence_stack() -> HashMap<char, i32> {
        let mut m = HashMap::new();
        m.insert('(', 0);
        m.insert('+', 2);
        m.insert('-', 2);
        m.insert('*', 4);
        m.insert('/', 4);
        m.insert('n', 5); // 'n' represents 'neg'
        m
    }

    fn precedence_input() -> HashMap<char, i32> {
        let mut m = HashMap::new();
        m.insert('(', 6);
        m.insert(')', 0);
        m.insert('+', 1);
        m.insert('-', 1);
        m.insert('*', 3);
        m.insert('/', 3);
        m.insert('n', 5); // 'n' represents 'neg'
        m
    }
}

```

```
}
```

```
fn operators() -> Vec<char> {  
    vec!['+', '-', '*', '/']  
}
```

```
fn transform(&self, s: String) -> Vec<String> {  
    let mut output: Vec<String> = Vec::new();  
    let mut s = s;  
    s.push(')');  
    let mut stack: Vec<char> = vec!['('];  
    let mut prev_c = '(';  
    let mut has_num = false;  
    let mut num = 0;  
    for c in s.chars() {  
        if c == ' ' {  
            continue;  
        }  
        if c.is_digit(10) {  
            has_num = true;  
            num = num * 10 + c.to_digit(10).unwrap() as i32;  
            prev_c = c;  
            continue;  
        }  
        if has_num {  
            has_num = false;  
            output.push(num.to_string());  
            num = 0;  
        }  
        let mut current_c = c;  
        if c == '-' && (prev_c == '(' || Self::operators().contains(&prev_c)) {  
            current_c = 'n';  
        }  
        while Self::precedence_stack()[stack.last().unwrap()] >= Self::precedence_input()[&c] {  
            let popped = stack.pop().unwrap();  
            if popped != '(' {  
                output.push(popped.to_string());  
            } else {  
                break;  
            }  
        }  
        stack.push(current_c);  
    }  
    output
```

```

    }

    if current_c != ')' {
        stack.push(current_c);
    }

    prev_c = c;
}

output
}

```

```

fn evaluate_postfix(&self, postfix: Vec<String>) -> i32 {
    let mut stack: Vec<i32> = Vec::new();
    for each in postfix {
        if let Ok(num) = each.parse::<i32>() {
            stack.push(num);
            continue;
        }
        match each.as_str() {
            "+" => {
                let a = stack.pop().unwrap();
                let b = stack.pop().unwrap();
                stack.push(a + b);
            }
            "-" => {
                let a = stack.pop().unwrap();
                let b = stack.pop().unwrap();
                stack.push(b - a);
            }
            "*" => {
                let a = stack.pop().unwrap();
                let b = stack.pop().unwrap();
                stack.push(a * b);
            }
            "/" => {
                let a = stack.pop().unwrap();
                let b = stack.pop().unwrap();
                stack.push(b / a);
            }
            "n" => {
                let a = stack.pop().unwrap();
                stack.push(-a);
            }
        }
    }
}

```

```

    }
    _ => panic!("Unknown operator"),
}
}
stack[0]
}

pub fn calculate(s: String) -> i32 {
    let postfix = Solution.transform(&Solution, s);
    Solution.evaluate_postfix(&Solution, postfix)
}
}

```

There is a compile error: Char 1: error: the name `Solution` is defined multiple times (solution.rs) | 3 | struct Solution; | ----- previous definition of the type `Solution` here. check the rust code and modify it to solve the error.

HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

rust

◇ 折叠复制

```

use std::collections::HashMap;

struct Solution;

impl Solution {
    fn precedence_stack() -> HashMap<char, i32> {
        let mut m = HashMap::new();
        m.insert('(', 0);
        m.insert('+', 2);
        m.insert('-', 2);
        m.insert('*', 4);
        m.insert('/', 4);
        m.insert('n', 5); // 'n' represents 'neg'
        m
    }

    fn precedence_input() -> HashMap<char, i32> {

```

```

let mut m = HashMap::new();

m.insert('(', 6);
m.insert(')', 0);
m.insert('+', 1);
m.insert('-', 1);
m.insert('*', 3);
m.insert('/', 3);
m.insert('n', 5); // 'n' represents 'neg'

m

}

fn operators() -> Vec<char> {
    vec!['+', '-', '*', '/']
}

fn transform(&self, s: String) -> Vec<String> {
    let mut output: Vec<String> = Vec::new();
    let mut s = s;
    s.push(')');
    let mut stack: Vec<char> = vec!['('];
    let mut prev_c = '(';
    let mut has_num = false;
    let mut num = 0;
    for c in s.chars() {
        if c == ' ' {
            continue;
        }
        if c.is_digit(10) {
            has_num = true;
            num = num * 10 + c.to_digit(10).unwrap() as i32;
            prev_c = c;
            continue;
        }
        if has_num {
            has_num = false;
            output.push(num.to_string());
            num = 0;
        }
        let mut current_c = c;
        if c == '-' && (prev_c == '(' || Self::operators().contains(&prev_c)) {

```

```

        current_c = 'n';
    }

    while Self::precedence_stack()[stack.last().unwrap()] >= Self::precedence_input()[&c] {
        let popped = stack.pop().unwrap();
        if popped != '(' {
            output.push(popped.to_string());
        } else {
            break;
        }
    }

    if current_c != ')' {
        stack.push(current_c);
    }

    prev_c = c;
}

output
}

```

```

fn evaluate_postfix(&self, postfix: Vec<String>) -> i32 {
    let mut stack: Vec<i32> = Vec::new();
    for each in postfix {
        if let Ok(num) = each.parse::<i32>() {
            stack.push(num);
            continue;
        }
        match each.as_str() {
            "+" => {
                let a = stack.pop().unwrap();
                let b = stack.pop().unwrap();
                stack.push(a + b);
            }
            "-" => {
                let a = stack.pop().unwrap();
                let b = stack.pop().unwrap();
                stack.push(b - a);
            }
            "*" => {
                let a = stack.pop().unwrap();
                let b = stack.pop().unwrap();
                stack.push(a * b);
            }
        }
    }
}

```

```

        "/" => {
            let a = stack.pop().unwrap();
            let b = stack.pop().unwrap();
            stack.push(b / a);
        }
        "n" => {
            let a = stack.pop().unwrap();
            stack.push(-a);
        }
        _ => panic!("Unknown operator"),
    }
}

stack[0]
}

pub fn calculate(s: String) -> i32 {
    let solution = Solution;
    let postfix = solution.transform(s);
    solution.evaluate_postfix(postfix)
}
}

```

The LeetCode Rust submission failed with the following error. Please fix the code accordingly.

```
Line 140: Char 1: error: the name Solution is defined multiple times
(solution.rs) | 3 | struct Solution; | ----- previous definition of the type
Solution here ... 140 | struct Solution; | ^^^^^^^^^^^^^^^^^^^^^ Solution
redefined here | = note: Solution must be defined only once in the type
namespace of this module
Line 115: Char 32: error: this method takes 1 argument
but 2 arguments were supplied (solution.rs) | 115 | let postfix =
Solution.transform(&Solution, s); | ^^^^^^^^^^ ----- unexpected argument #1
of type &Solution |
Line 33: Char 8: note: method defined here (solution.rs) | 33 | fn
transform(&self, s: String) -> Vec<String> { | ^^^^^^^^^^ help: remove the extra
argument | 116 - let postfix = Solution.transform(&Solution, s); 116 + let postfix =
Solution.transform(s); |
Line 116: Char 18: error: this method takes 1 argument
but 2 arguments were supplied (solution.rs) | 116 | Solution.evaluate_postfix(&Solution,
postfix) | ^^^^^^^^^^^^^^^^^^^^^ ----- unexpected argument #1 of type
&Solution |
Line 76: Char 8: note: method defined here (solution.rs) | 76 | fn
evaluate_postfix(&self, postfix: Vec<String>) -> i32 { | ^^^^^^^^^^^^^^^^^^^^^ help:
```

remove the extra argument | 117 - Solution.evaluate_postfix(&Solution, postfix) 117
+ Solution.evaluate_postfix(postfix) | Some errors have detailed explanations: E0061,
E0428. For more information about an error, try `rustc --explain E0061` . error: could
not compile `prog` (bin "prog") due to 3 previous errors

HKGAI-V1 | Free

- -- Free credit: 8.0; Paid credit: 0.0 --

rust

折叠复制

```
use std::collections::HashMap;

struct Solution;

impl Solution {
    fn precedence_stack() -> HashMap<char, i32> {
        let mut m = HashMap::new();
        m.insert('(', 0);
        m.insert('+', 2);
        m.insert('-', 2);
        m.insert('*', 4);
        m.insert('/', 4);
        m.insert('n', 5); // 'n' represents 'neg'
        m
    }

    fn precedence_input() -> HashMap<char, i32> {
        let mut m = HashMap::new();
        m.insert('(', 6);
        m.insert(')', 0);
        m.insert('+', 1);
        m.insert('-', 1);
        m.insert('*', 3);
        m.insert('/', 3);
        m.insert('n', 5); // 'n' represents 'neg'
        m
    }

    fn operators() -> Vec<char> {
        vec!['+', '-', '*', '/']
    }
}
```

```

fn transform(&self, s: String) -> Vec<String> {
    let mut output: Vec<String> = Vec::new();
    let mut s = s;
    s.push(' ');
    let mut stack: Vec<char> = vec!['('];
    let mut prev_c = '(';
    let mut has_num = false;
    let mut num = 0;
    for c in s.chars() {
        if c == ' ' {
            continue;
        }
        if c.is_digit(10) {
            has_num = true;
            num = num * 10 + c.to_digit(10).unwrap() as i32;
            prev_c = c;
            continue;
        }
        if has_num {
            has_num = false;
            output.push(num.to_string());
            num = 0;
        }
        let mut current_c = c;
        if c == '-' && (prev_c == '(' || Self::operators().contains(&prev_c)) {
            current_c = 'n';
        }
        while Self::precedence_stack()[stack.last().unwrap()] >= Self::precedence_input()[&c] {
            let popped = stack.pop().unwrap();
            if popped != '(' {
                output.push(popped.to_string());
            } else {
                break;
            }
        }
        if current_c != ')' {
            stack.push(current_c);
        }
        prev_c = c;
    }
}

```

```

    }
    output
}

```

```

fn evaluate_postfix(&self, postfix: Vec<String>) -> i32 {
    let mut stack: Vec<i32> = Vec::new();
    for each in postfix {
        if let Ok(num) = each.parse::<i32>() {
            stack.push(num);
            continue;
        }
        match each.as_str() {
            "+" => {
                let a = stack.pop().unwrap();
                let b = stack.pop().unwrap();
                stack.push(a + b);
            }
            "-" => {
                let a = stack.pop().unwrap();
                let b = stack.pop().unwrap();
                stack.push(b - a);
            }
            "*" => {
                let a = stack.pop().unwrap();
                let b = stack.pop().unwrap();
                stack.push(a * b);
            }
            "/" => {
                let a = stack.pop().unwrap();
                let b = stack.pop().unwrap();
                stack.push(b / a);
            }
            "n" => {
                let a = stack.pop().unwrap();
                stack.push(-a);
            }
            _ => panic!("Unknown operator"),
        }
    }
    stack[0]
}

```

```
}  
  
pub fn calculate(s: String) -> i32 {  
    let solution = Solution;  
    let postfix = solution.transform(s);  
    solution.evaluate_postfix(postfix)  
}  
}
```

